IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 5 September 2025, accepted 3 October 2025, date of publication 13 October 2025, date of current version 17 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3620431

APPLIED RESEARCH

Automatic Test Case Generation Mechanism With Natural Language-Based Korean Requirement Specifications

WOOSUNG JANG AND R. YOUNG CHUL KIM
SE Laboratory, Hongik University, Sejong-si 30016, Republic of Korea

Corresponding author: R. Young Chul Kim (bob@hongik.ac.kr)

This work was supported in part by Korea Creative Content Agency (KOCCA) Grant funded by the Ministry of Culture, Sports and Tourism (MCST), in 2025 through the Project "Artificial Intelligence-Based User Interactive Storytelling 3-D Scene Authoring Technology Development," under Project RS-2023-00227917307820870502O1; and in part by Korea Research Foundation's Brain Korea 21 Fostering Outstanding Universities for Research (BK21 FOUR) funded by the Ministry of Education (MOE), in 2025, through the Project "Ultra-Distributed Autonomous Computing Service Technology Research Team," under Project 202003520006.

ABSTRACT In the software industry, software testers are forced to make test cases with informal or formal requirements. From a software perspective, to conduct user acceptance testing, we should manually create numerous test cases to achieve the desired test coverage based on a precise analysis of clear and unstructured requirements. Until now, defining and analyzing the meaning of natural language-based requirements has been challenging, and automating this process has also been complex. In software engineering, rare research generates test cases with natural language requirement sentences, especially in Korea. To solve this problem, we propose a mechanism for automatically generating test cases from natural language requirements, as used by software engineers. This approach involves 1) simplifying unstructured Korean requirements, 2) normalizing requirement sentences, 3) modeling the requirement simplification process (C3Tree modeling), 4) automatically generating cause-effect graph model, and then test cases via decision table model with the graph models through metamodel transformation methods, 5) semi-automatic generation of test scripts, 6) generating test result reports, and 7) finally automatic generation of requirement traceability matrices. Ultimately, simplifying unstructured Korean requirements sentences facilitates understanding of requirements among stakeholders and reduces testing costs. At this moment, we will compare the exact way in which the test case generation approach and generative AI-based test case generation are created.

INDEX TERMS Automatic test case generation, cause-effect graph, C3Tree model, decision table, Korean natural language requirements specifications, model-driven architecture (MDA).

I. INTRODUCTION

Artificial Intelligence (AI) approaches are now widely used for generating various things. In software engineering, developing automated test cases from software requirements is a crucial issue in the software field, especially when dealing with unstructured requirements [1]. Automating test case generation can improve the cost-effectiveness and accuracy of software testing. While significant research has been conducted on automatically generating test cases from source code or UML models, research is still lacking on automatically generating test cases from natural language-based requirements specifications [2], [3], [4], [5], [6]. In particular, research on generating test cases from Korean language requirements is rare [6]. A few scholars are researching this area, but fully automating the generation of natural language-based test cases remains challenging. Recent research has investigated test case generation through an AI techniques approach; however, validating the test case generation process is challenging [7]. Consequently, maintaining traceability between requirements and test cases—a critical factor in software testing—is difficult.

The associate editor coordinating the review of this manuscript and approving it for publication was Claudia Raibulet.

To solve this problem, we focus on analyzing Korean Natural Language requirements and aim to generate test cases [7] automatically. In our previous research, we investigated the generation of cause-and-effect graphs from Korean language requirements [8], [9]. However, prior research did not fully address the issue of automatic test case generation. Additionally, it did not consider problems related to negation in sentences, the identification of nested negations, and the separation of suffixes, which hinder the analysis of complex sentences. To address these issues, we propose a method for generating C3Tree models from Korean requirements, creating cause-effect graphs from these models, automatically transforming cause-effect graph models into decision table models, and subsequently generating test case models using the decision tables based on metamodel-based model transformation. Additionally, we propose inserting test scripts manually into the intermediate process to automatically execute test cases and generate requirement traceability matrices during this process. This method enables the modeling of the requirement analysis process and supports the easy creation, addition, deletion, and update of model information during the test case generation process.

We need to compare which approach is more accurate between our method and the AI LLM approach.

This paper is mentioned as follows. Chapter 2 describes related research. Chapter 3 describes the method for automatically generating test cases from Korean natural language requirements. Chapter 4 discusses experiments and results. Finally, we describe the conclusion.

II. RELATED RESEARCH

A. REQUIREMENTS-BASED TESTING USING CAUSE-EFFECT GRAPH

Requirements-driven testing aims to improve the quality of requirements and generate the minimum number of test cases that cover 100% of the requirements. A cause-and-effect graph can model all types of logic and represent the inputs (causes) and outputs (effects) of requirements as true or false. Traditional test case generation techniques based on a cause-and-effect graph can generate the minimum number of test cases that cover 100% of functional requirements through a decision table.

Fig. 1 shows the components of a cause-and-effect graph. A node is the smallest functional unit in a requirement. A node can have a value of either true or false. True means the successful execution of the function. False indicates the failure of the function. Each node can be connected to other nodes. The left node means the cause function, and the right node means the result function. The node relationship contains the operators 'Identify', 'NOT', 'AND', and 'OR'. The identity, where if the cause is actual, the effect is true. The NOT, where if the cause is false, the effect is true, and if the cause is actual, the effect is false. The AND, where if all causes are true, the effect is true. The OR, where if any one of the causes is true, the effect is actual.

Fig. 2 shows an example cause-and-effect graph for the sentence "If A is input or B is input, then C is output."

A decision table is a table that represents combinations of values for various elements. A cause-and-effect graph can generate test cases through a decision table. Each combination of nodes in a cause-and-effect graph is converted into a row in the decision table. Each combination (True/False) case of each node is transformed into a column in the decision table. The columns in the decision table become test cases. Table 1 shows the result of generating a decision table from the cause-effect graph of Fig. 2. Table 2 shows the test cases generated from the decision table.

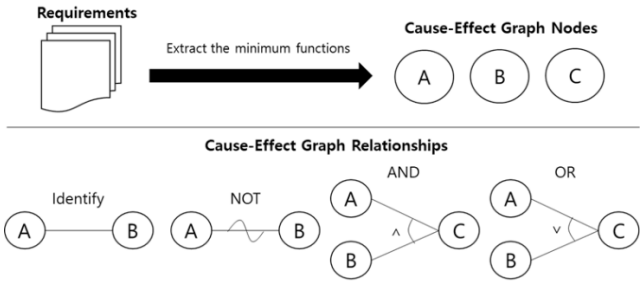


FIGURE 1. Components of a cause-and-effect graph.

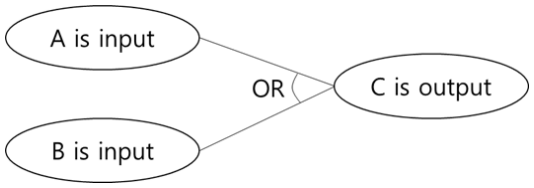


FIGURE 2. Example of a cause-and-effect graph.

TABLE 1. Example of a decision table.

	TEST #1	TEST #2	TEST #3	TEST #4
Causes:				
A is input	T	F	F	T
B is input	F	T	F	T
Effects:				
C is output	T	T	F	T

TABLE 2. Example of a test case.

TC	Cause	Effect
1	A is input B is not input	C is output
2	A is not input B is input	C is output
3	A is not input B is not input	-
4	A is input B is input	C is output

B. TEST CASE GENERATION FROM NATURAL LANGUAGE REQUIREMENT SENTENCES

The previous research on generating test cases based on natural language requirement sentences is as follows.

1) TEST CASE GENERATION USING TEXT MINING AND SYMBOL-BASED EXECUTION METHODOLOGIES

Elghondakly et al. [2] integrates nouns, verbs, adverbs, adjectives, and other elements from requirement sentences using text mining and symbol-based execution methodologies. It generates test paths, test cases, and test data from sentences. However, it does not consider generating as many test cases as possible and does not address the similarity between test cases. Additionally, it cannot predict test data and uses words from the sentence as test data.

2) CASDL-BASED TEST CASE GENERATION

Zheng et al. [3] generates test cases and test inputs that meet the MC/DC criterion from natural language requirements written in the Casco Accurate Specification Description Language (CASDL). It automatically executes test cases to derive test oracles. However, it cannot generate test cases from unstructured natural language and does not consider functional redundancy.

3) ACCEPTANCE TEST CASE GENERATION FROM USE CASE SPECIFICATION

Wang et al. [4] automatically generates acceptance test cases from use case specifications, identifies test scenarios, and generates execution and constraint conditions. It considers the semantic similarity of sentences. However, it lacks consideration for analyzing unstructured natural language requirements.

4) TEST CASE GENERATION USING KEYWORDS IN THE REQUIREMENT SENTENCES

Ansari et al. [5] proposes a method for automatically generating test cases from functional requirement sentences based on keywords in the requirement sentences. However, it has limited identifiable keywords and does not consider sentence redundancy.

5) TEST CASE GENERATION BASED ON KOREAN MORPHEMES

Han et al. [6] analyses the morphemes of Korean sentences to generate test cases and test scripts. However, this paper can only analyze standardized requirements. It imposes many constraints on recognizable sentence structures, such as requiring active voice, short sentences, and the inclusion of '경우(Kyeongwoo)' (which means 'in case of').

III. AUTOMATIC TEST CASE GENERATION METHOD WITH KOREAN NATURAL LANGUAGE REQUIREMENTS

We propose a process for automatically generating test cases from unstructured Korean natural language requirements sentences. For restructuring with requirement sentences, we define C3Tree models to transform cause-effect graphs. With this graph, we transform decision tables to generate test cases. Finally, we manually insert test scripts in the intermediate process to perform automated testing.

Fig. 3 shows the overall process. This process builds upon existing research [8], [9], with Steps 1, 2, and 4 remaining consistent with previous work. Step 3 has been improved upon from prior research, while Steps 5 to 8 represent new research findings.

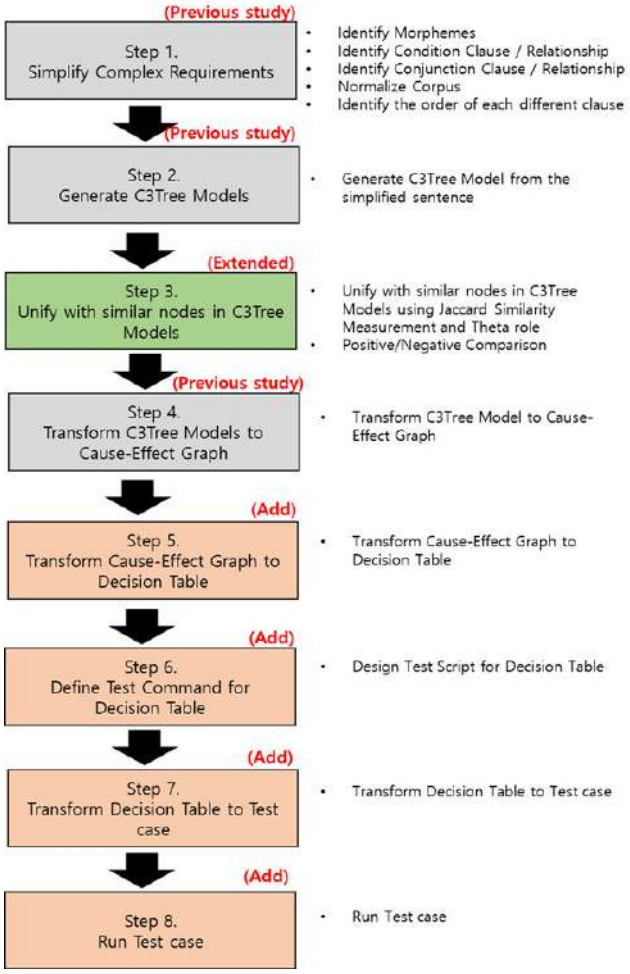


FIGURE 3. Automatic test case generation process from Korean language-based Requirement specifications.

A. STEP 1. SIMPLIFY COMPLEX REQUIREMENTS

In previous studies, we analyzed the morphemes of natural language requirement sentences to generate a cause-and-effect graph. This method divides complex sentences into simple clauses. The relationships between simple clauses are distinguished as if-then, AND, OR, and NOT.

The Connective Ending morpheme in Korean is a unique morpheme that does not exist in English. It is located at the end of a clause, connecting the previous clause with the following clause or ending a sentence. We distinguish Connective Endings as Conditional Connective Endings or Conjunctive Connective Endings. Conditional Connective Endings are used to connect clauses in a cause-and-effect

relationship. Conjunctive Connective Endings are used to connect clauses in a connection relationship.

We distinguish clauses containing conditional connective endings as 'If-then' and 'If-then not' relationships, and also conjunctive connective endings as AND or OR. Table 3 and Fig. 4 show the results of identifying conditional and connective sentences in natural language sentences.

TABLE 3. Requirement specification 1 (R1).

Category	Example
Korean	아이디/비밀번호가 입력되고, 유효하지 않다면, 시 스템에 의해 로그인 정보가 재요청된다.
Pronunciation	aidi/bimilbeonhoga iblyeogdoego, yuhyohaji anhdamyeon, siseutem-e uihae logeu-in jeongboga jaeyocheongdoenda.
English	If the ID/Password is entered incorrectly, the system requests the login information again.

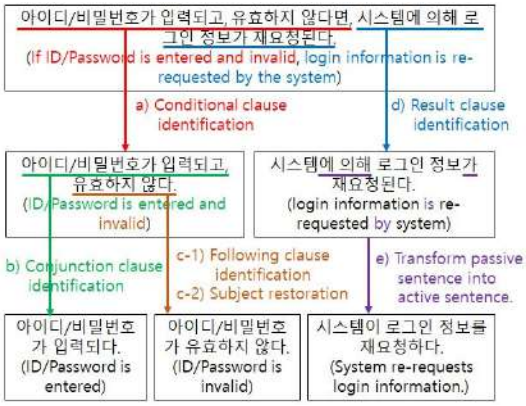


FIGURE 4. A simplification process in R1.

B. STEP 2. GENERATE C3TREE MODELS

We use the algorithm of Step 1 to identify If-then, if-then not, AND, OR relationships from natural language sentences to create a C3Tree Model. The C3Tree Model represents the process of separating clauses from sentences as a binary tree structure. Tables 4 and 5 describe the node types and relationships of the C3Tree Model.

TABLE 4. Nodes of the C3Tree model.

Node	Description
<<Sentence>> Korean Sentence	Root node. It contains the complex structure of the original sentence.
<<Complex-Clause>> Korean Sentence	Intermediate node. It is split from <<Sentence>> and contains sentences that can be further split.
<<Clause>> Korean Sentence	Leaf node. It contains the most simplified sentences (short sentences).

Fig. 5 shows the simplification process of natural language sentences using the C3Tree Model. The process for creating the C3Tree Model for Requirement Sentence 1 (R1) is as follows: 1) Identify the cause clause and effect clause from R1. 2) Identify conjunction clauses and subsequent clauses (AND) within the cause clause. 3) Represent the original sentence as a ≪Sentence≫ node. 4) Represent the cause clause as a ≪Complex-Clause≫ node. 5) Represent the result clause, conjunction clause, and subsequent clause as ≪Clause≫ nodes. 6) Express the relationships between each node.

TABLE 5. Relationships of the C3Tree model.

Node	Description
COND-P	Child nodes are positive conditional relationships.
COND-N	Child nodes are negatively conditional relationships.
CONJ-AND	Child nodes are conjunction(AND) relationships.
CONJ-OR	Child nodes are conjunction(OR) relationships.

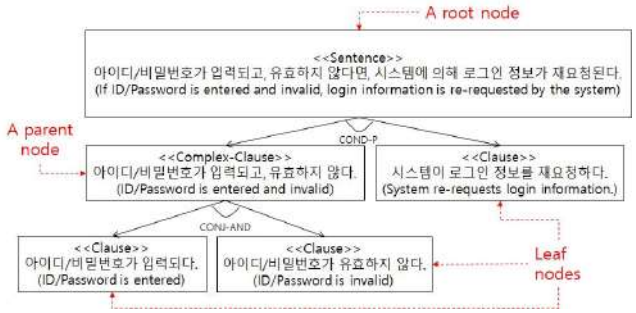


FIGURE 5. C3Tree model of R1.

C. STEP 3. UNIFY WITH SIMILAR NODES IN C3TREE MODELS

A single requirement sentence is generated into one C3Tree model. In Step 3, multiple C3Tree models are unified by merging leaf nodes with identical meanings. In prior research [8], Step 3 measured the similarity between nodes using Jaccard similarity [13] and semantic roles [14]. The Jaccard similarity measures the similarity between sets A and B, yielding a value between 0 and 1.0. The formula for Jaccard similarity is shown in (1). Jaccard similarity is used to measure the morpheme similarity of each node in the C3Tree Model.

$$J(A, B) = \frac{A \cap B}{A \cup B} \tag{1}$$

Table 6 shows an example requirement sentence(R2). Fig. 6 shows the Jaccard similarity between the leaf node of R1 and the leaf node of R2 when calculated. During morpheme comparison, case markers(Only Korean morphemes)

are removed, and the final consonants of inflectional endings are removed before contrast. In this example, the case markers 이(i)' and '를(leul)' are removed, and the final consonant 'ㄴ(n)' from R2's inflectional ending '한(han)' is removed. As a result, the Jaccard similarity between the red box nodes is 1.0.

TABLE 6. Requirement specification 2 (R2).

Category	Example
Korean	로그인 세션이 만료되면, 시스템이 로그인 정보를 재요청한다.
Pronunciation	logeu-in sesyeon-i manlyodoemyeon, siseutem-i logeu-in jeongboleul jaeyocheonghanda.
English	If the login session expires, the system re-requests login information.

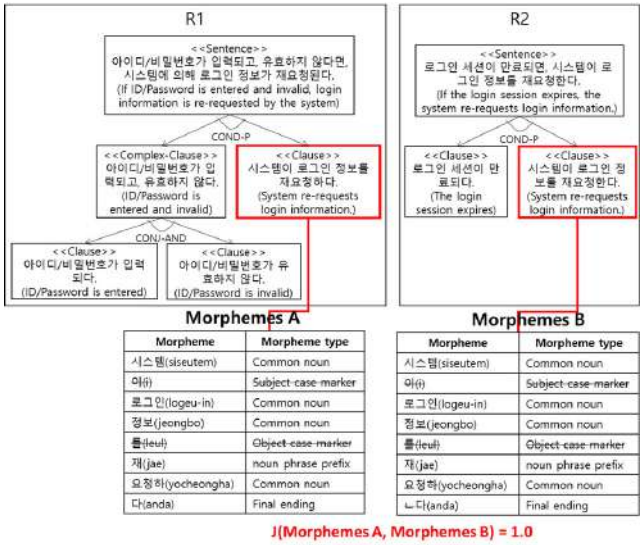


FIGURE 6. Compare the morphemes of two nodes.

Even if the Jaccard similarity between nodes is above 0.8, sentence meanings can differ. To solve this problem, theta roles are additionally identified to combine nodes with overlapping meanings. Theta roles indicate the roles of participants in semantic structures related to predicates. We analyze theta roles using the Exobrain library from the Electronics and Telecommunications Research Institute (ETRI) [14]. Table 7 shows part of the Sejong Electronic Dictionary's theta roles applied to the Exobrain library.

We use Exobrain to compare all theta roles of all ≪Clause≫s. Fig. 7 shows the ≪Clause≫ nodes with the same semantic roles between R1 and R2. In the two ≪Clause≫, the same theme and predicate were identified, and no other theta roles were identified.

However, prior research [8] performs inaccurate sentence comparisons when negations or suffixes are present. To address this problem, this paper proposes methods for negation comparison, nested negation identification, and

TABLE 7. Part of the Sejong electronic dictionary's theta roles.

Theta-role	Description
Agent	An object that causes an action with the intention expressed by the predicate.
Experience	The entity that recognizes an action or a state, not causing the action with the intention.
Patient	The person or thing that undergoes the action.

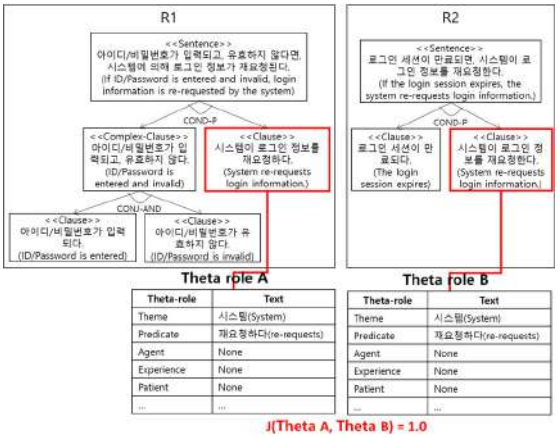


FIGURE 7. Compare the theta-roles of two nodes.

morpheme comparison with suffixes. If a sentence contains a negation adverb, it conveys a negative meaning. For example, "공부를 하다(gongbuleul hada)" (to study) and "공부를안하다 (gongbuleul anhada)" (not to study) have similar morphemes but convey opposite meanings. Therefore, even with a high Jaccard similarity, sentences with negations should be identified as having different meanings. Additionally, sentences can contain multiple nested negative meanings. In most cases, if negations are nested twice, they are interpreted as having a positive sense. Therefore, sentences with an even number of nested negations are transformed into positive sentences, while those with an odd number of nested negations are converted into a single negation.

We use a morphological analyzer to identify negative morphemes in natural language sentences and analyze whether the sentence is negative. If there are an even number of negative morphemes modifying a verb, it is judged as positive, and if there are an odd number, it is considered as negative. This method can generate more accurate 'if-then not' relationships in the C3Tree model.

Furthermore, suffixes are removed from verbs in sentences. For example, in Korean, verbs like 열리다(yeollida)"(to be opened) have a form combining a verb and an inflectional ending. The inflectional ending is responsible for converting the verb into a passive form and is not automatically analyzed by Korean morpheme analyzers. "열리다(yeollida)" (to be opened) and "열다(yeolda)" (to open) may be recognized as different morphemes by a morpheme analyzer, although

they have the same meaning. In such cases, we identify "열-(yeol)" as a verb and "리(li)" as a suffix, separate them, and delete "리(li)," ultimately resulting in "열다(yeolda)" (to open).

As a result, only critical morphemes such as subjects, verbs, and objects are compared, and sentences with a similarity of 0.8 or higher are considered to have similar morphemes. Fig. 8 shows the integration of C3Tree models for R1 and R2. One node has been integrated.

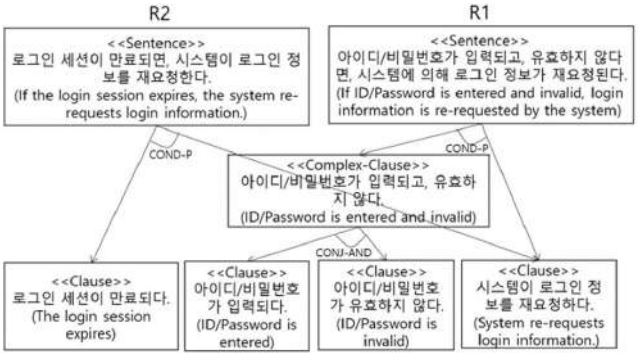


FIGURE 8. Integration of the R1 model and the R2 model.

From this point onwards, Korean grammar is not discussed. We only mention the model's transformation. Therefore, for ease of understanding, we will translate all Korean sentences into English sentences for explanation. Fig. 9 is the result of Fig. 8 being translated into English.

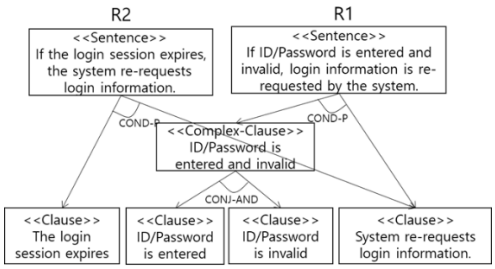


FIGURE 9. The model is expressed in english.

D. STEP 4. TRANSFORM C3TREE MODELS TO CAUSE-EFFECT GRAPH

Step 4 generates a cause-and-effect graph model from the integrated C3Tree model. The components of the integrated C3Tree model are transformed into cause-and-effect graph components. Table 8 shows the relationships between the two models. For example, the ≪Clause≫ in the C3Tree model is transformed into a Node in the Cause-Effect Graph. Fig. 10 shows the generation of a cause-and-effect graph from the integrated C3Tree model. The four ≪Clause≫ nodes at the lowest level of the C3Tree are transformed into nodes A to D in the cause-effect graph. The left node of COND-P is transformed into a cause node, while the right node of COND-P is transformed into an effect node. CONJ-AND is converted into an AND relationship.

TABLE 8. Relationship between C3Tree model notation and cause-effect graph notation.

Cases	C3Tree Model Notation	Cause-Effect Graph Notation
1	<<Clause>>	Node
2	CONJ-AND	AND
3	CONJ-OR	OR
4	The left child node of COND	Cause
5	The right child node of COND	Effect
6	COND-P	Identity
7	COND-N	NOT

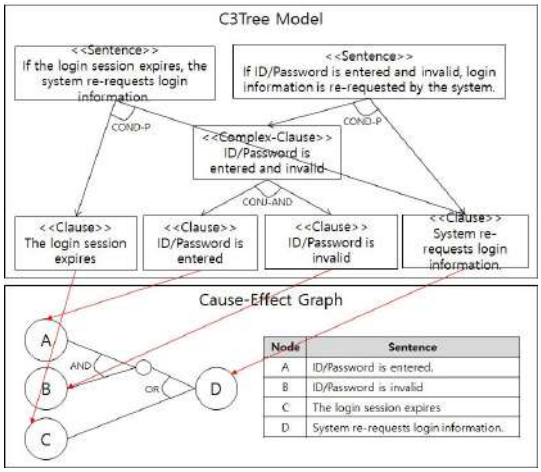


FIGURE 10. Cause-effect graph generated from the integrated C3Tree model.

E. STEP 5. TRANSFORM CAUSE-EFFECT GRAPH TO DECISION TABLE

Step 5 transforms the cause-effect graph into a decision table using the Model Driven Architecture (MDA) [15] metamodel-based model transformation technique. This method supports easily adding, modifying, and deleting model transformation rules.

Fig. 11 illustrates the metamodel of the cause-effect graph, which generates a cause-effect graph model based on this metamodel. It demonstrates the relationships between the cause-effect graph metamodel, the XMI code of the cause-effect graph model, and the visualized cause-effect graphs, as indicated by red arrows. The top of Fig. 11 shows the cause-and-effect graph metamodel. It consists of 'CauseEffect,' 'Connector,' 'Element,' 'Cause,' and 'Effect' classes. The 'CauseEffect' includes multiple 'Connector' and 'Element' pieces of information. The 'Element' contains information about the nodes in the cause-and-effect graph. The 'ceid' is the unique identifier of an 'Element.' The 'Value' represents the natural language sentence. The 'action,' the 'actor,' the 'target,' and the 'beneficiary' are the theta roles of the sentence. The 'lastSyntax' is the last verb in the sentence from syntactic analysis. The 'Meaning'

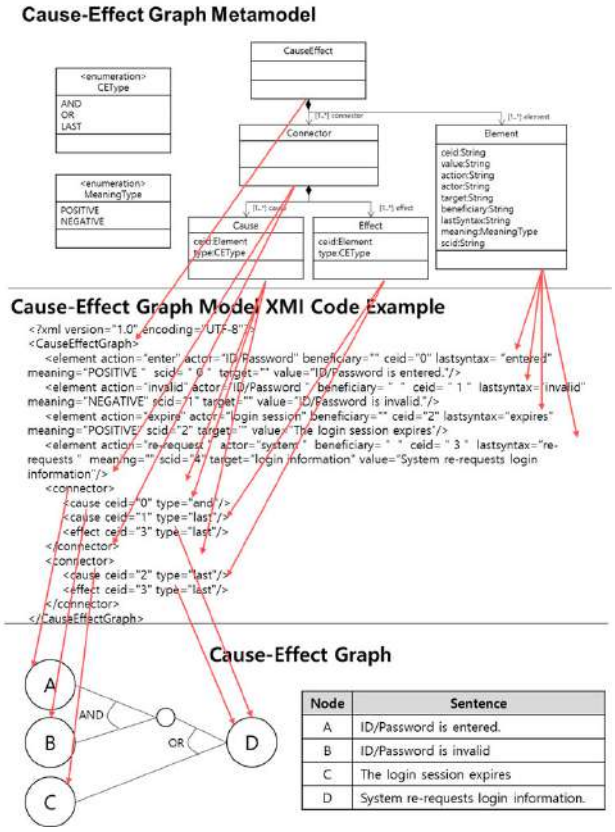


FIGURE 11. Process of generating a cause-effect graph model based on the cause-effect graph metamodel.

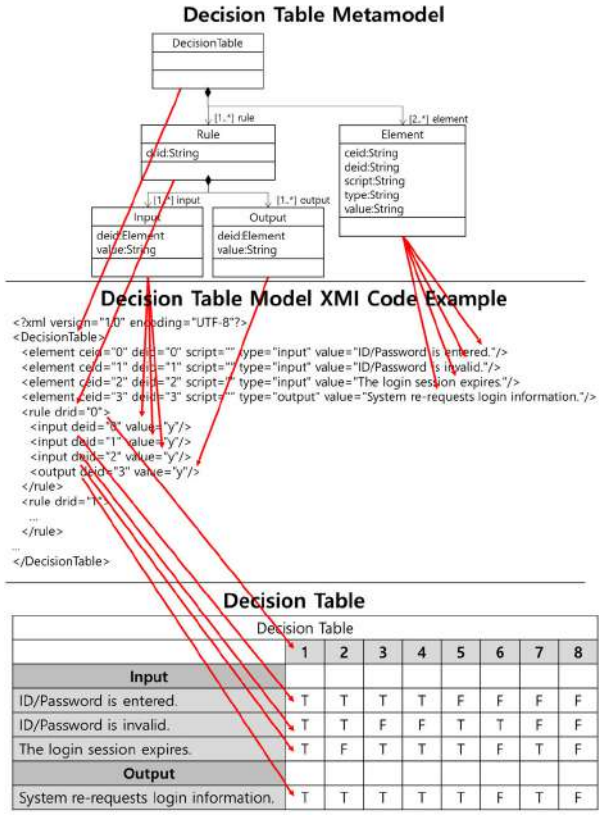


FIGURE 12. Process of generating a decision table model based on the decision table metamodel.

indicates whether the sentence is negated or not. The 'scid' is the unique identifier of the original sentence. The 'Connector' contains 'Cause' and 'Effect' related by the 'Connector.' The 'Cause' includes information on the Cause-related 'Element.' The 'ceid' is the 'Element' number. The 'Type' is one of the 'CEType' values. The 'AND' indicates an AND relationship with the next node. The 'OR' indicates an OR relationship with the next node. The 'LAST' signifies that this is the last clause. The 'Effect' contains information about the Effect-related 'Element.' The 'ceid' is the Element number. The 'Type' is one of the 'CEType' values. The 'Effect' only uses LAST as an attribute value.

The middle of Fig. 11 shows an example XMI code of the cause-effect graph generated based on the cause-effect graph metamodel. The 'CauseEffectGraph' tag contains four 'element' tags and two 'connector' tags. Each 'connector' tag contains n 'cause' tags and one 'effect' tag. The 'cause' tag includes information on AND or OR relationships with the following 'cause' tags. The bottom of Fig. 11 shows the result of expressing an XMI code as a cause-and-effect graph. The four 'element' tags are represented as nodes, and the two 'connector' tags represent pairs of cause-effect nodes. The 'cause' tag is represented as a cause node, and the 'effect' tag is represented as an effect node. In addition, we designed metamodels for decision tables and test cases. Each model can be represented as XMI code.

Fig. 12 shows the decision table model and XMI code generated based on the decision table metamodel. The decision table metamodel consists of 'DecisionTable,' 'Rule,' 'Input,' 'Output,' and 'Element.' The 'DecisionTable' contains multiple 'Rules' and 'Elements.' The 'Element' contains all the 'Input' and 'Output' values of the decision table.

The 'ceid' is the ID of the cause-effect graph node before being converted to a decision table. The 'deid' is a unique identifier of the 'Element.' The 'script' refers to a test script used during test execution. The 'type' means the 'Input' or 'Output' property of the decision table. The 'value' is a natural language sentence. The 'Rule' contains multiple 'Inputs' and 'Outputs.' The 'drid' is a unique identifier of the 'Rule.' The 'Input' contains input information. The 'deid' is a unique identifier—the 'value' stores True or False values. The 'Output' contains output information. The meaning of each attribute is the same as that of the 'Input.'

Fig. 13 illustrates the process of transforming the cause-effect graph into a decision table through model transformation based on a metamodel. The metamodel transformation engine, located in the middle of Fig. 13, reads the cause-effect graph model generated based on the cause-effect graph metamodel, references transformation rules, and generates the decision table model based on the decision table metamodel. The metamodel transformation engine is a program. The cause-effect graph metamodel, cause-effect graph, decision table metamodel, and decision table are all represented as

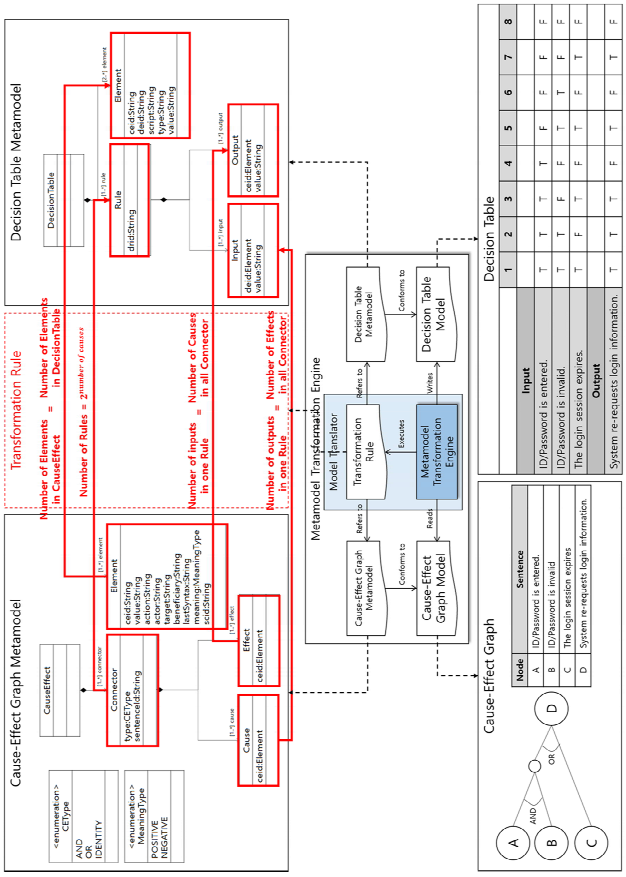


FIGURE 13. Process of transforming a cause-and-effect graph into a decision table using a metamodel transformation technique.

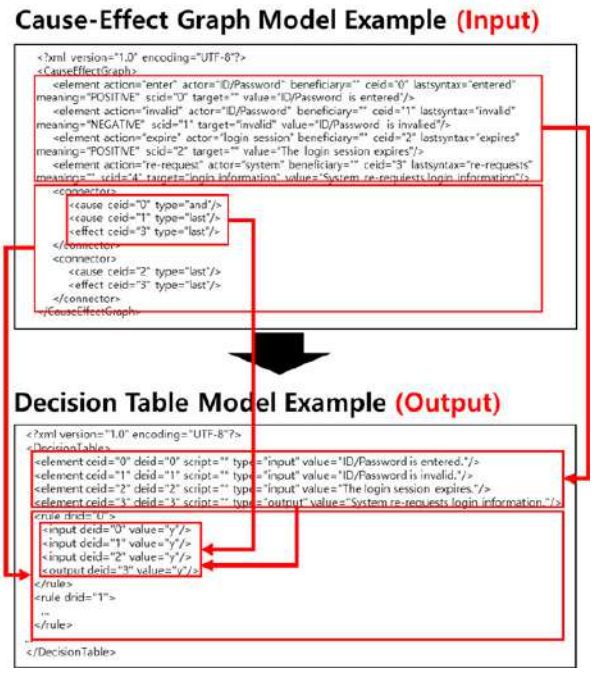


FIGURE 14. Metamodel-based model transformation process.

XMI files. Transformation rules are ATLAS Transformation Language (ATL) [16] files.

The red arrows in the upper part of Fig. 13 represent transformation rules between the cause-effect graph metamodel and the decision table metamodel. Connector is transformed into a Rule. The number of Rules is 2 to the power of the number of Causes. Element is transformed into Element. Cause is transformed into Input. The number of Inputs included in a Rule is the total number of Causes. The effect included in a Rule is transformed into Output. The number of Outputs is the total number of Effects. The bottom of Fig. 13 visualizes that the cause-effect graph from Table 11 has been automatically transformed into the decision Table 10 MI code.

Fig. 14 is the result of the metamodel transformation engine automatically transforming the cause-effect graph model into the decision table model by referencing the cause-effect graph metamodel and the decision table metamodel and reading the transformation rules. The Elements of Cause-Effect Graph were transformed into the Elements of Decision Table. Connectors were transformed into Rules. Causes and Effects were transformed into Inputs and Outputs. Inputs and Outputs include all Elements of the decision table.

F. STEP 6. DEFINE TEST COMMAND FOR DECISION TABLE

In Step 6, test commands for generating test scripts are inserted into the decision table. This information is stored in the 'script' attribute within the Element class of the decision table metamodel. Only in this part do we require the user to insert the script manually. The red text in Table 9 indicates the location where the script should be inserted.

In subsequent steps, when test cases are generated and testing is conducted, the test program communicates bidirectionally with the target program using scripts. A script is a predefined keyword that connects a test runner (KRA tester) to a target program. The KRA tester is software that generates and executes test cases. The target program is the software that implements the functions in the requirements. To automatically execute tests using scripts, the target program must implement a function that receives and processes the script.

If the Element's type is 'input,' the test program sends the script to the target program. If the Element's type is 'output,' the target program sends the test result script to the test program. Table 9 shows the Result of inserting scripts into the decision Table 10 MI code.

Fig. 15 shows a decision table with the script inserted.

Decision Table	1	2	3	4	5	6	7	8
Input								
ID/Password is entered. (inputIDPW)	T	T	T	T	F	F	F	F
ID/Password is invalid. (invalidIDPW)	T	T	F	F	T	T	F	F
The login session expires. (expiredLOGIN)	T	F	T	F	T	F	T	F
Output								
System re-requests login information. (rerequireLOGIN)	T	T	T	F	T	F	T	F

FIGURE 15. Input the test command into the decision table.

TABLE 9. Decision Table XMI code with script input.

XMI code with script input
<?xml version="1.0" encoding="UTF-8"?> <DecisionTable> <element ceid="0" deid="0" script="inputIDPW" type="input" value="ID/Password is entered."/> <element ceid="1" deid="1" script="invalidIDPW" type="input" value="ID/Password is invalid."/> <element ceid="2" deid="2" script="expiredLOGIN" type="input" value="The login session expires."/> <element ceid="3" deid="3" script="rerequireLOGIN" type="output" value="System re-requests login information."/> ... </DecisionTable>

G. STEP 7. TRANSFORM DECISION TABLE TO TEST CASE

Fig. 16 illustrates the process of transforming the decision table into test cases via metamodel-based model transformation. The metamodel transformation engine reads the decision table model generated based on the decision table metamodel, references transformation rules, and creates the test case model based on the test case metamodel.

The top right of Fig. 16 shows the structure of the test case metamodel. The test case metamodel includes 'Testcase,' 'Case,' 'Precondition,' 'Send,' and 'Recv' classes. The 'Testcase' contains multiple pieces of 'Case' information. The 'Case' contains multiple 'Precondition,' 'Send,' and 'Recv' for a test case. The 'drid' attribute means the unique identifier of an Element within the decision table. The 'Precondition' refers to the conditions that must be met for the test case to execute. It identifies the combinations that need to be executed in advance from the decision table. The 'script' attribute contains script code to be sent to the target program, and the 'value' means the natural language sentence. The 'Send' contains information sent by the test program to the target program. Each attribute is the same as 'Precondition.' The 'Recv' contains information received by the test program from the target program. Each attribute is the same as 'Precondition.'

The middle top part of Fig. 16 shows the transformation process between the decision table metamodel and the test case metamodel. The rule is transformed into a Case. A precondition is generated from one cause-effect node within the decision table and is associated with Both Input and Output. If the Input value of this item is actual, it creates a combination of Inputs that makes the Output of this item true as well. The input and script related to the Element are transformed into Send. Output and the script related to the Element are transformed into Recv.

The bottom part of Fig. 16 shows the Result of transforming the decision table into test cases. All rules are transformed into test cases. Rules with all combinations being false are not transformed.

Table 10 shows an example of XMI code for test cases in Fig. 16. The Testcase includes seven case tags. Each case contains multiple send and receive information.

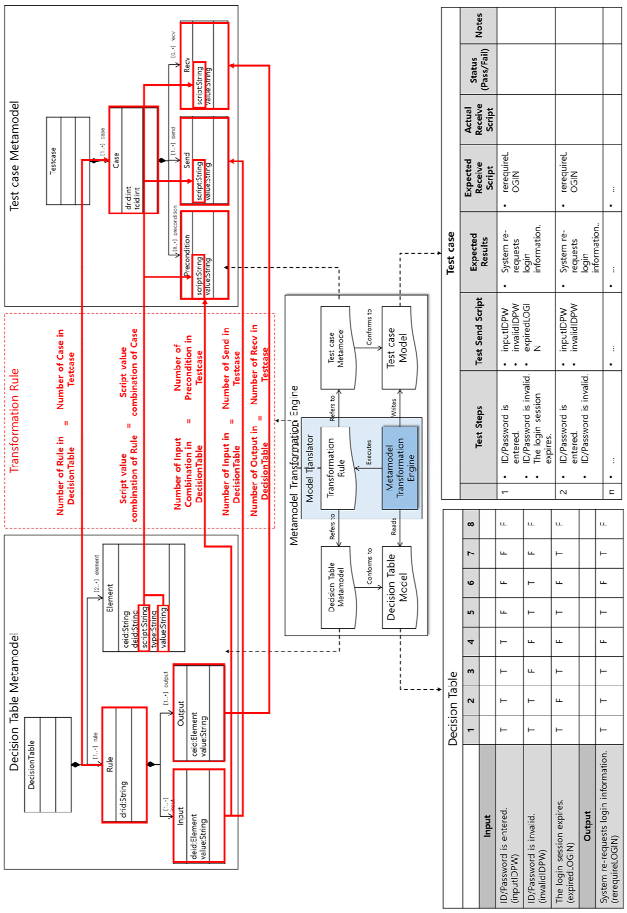


FIGURE 16. Process of transforming a decision table into a test case using a metamodel transformation technique.

TABLE 10. Test case XMI code with script input.

XMI code with script input
<?xml version="1.0" encoding="UTF-8"?> <Testcase> <case drid="0" tcid="0"> <send script="inputIDPW" value="ID/Password is entered."/> <send script="invalidIDPW" value="ID/Password is invalid."/> <send script="expiredLOGIN" value="The login session expires."/> <recv script="rerequireLOGIN" value="System re-requests login information."/> </case> <case drid="1" tcid="1"> <send script="inputIDPW" value="ID/Password is entered."/> <send script="invalidIDPW" value="ID/Password is invalid."/> <recv script="rerequireLOGIN" value="System re-requests login information."/> </case> ... </Testcase>

H. STEP 8. RUN TEST CASE

Step 8 automates the execution of tests. The test case's script is delivered to the target program for execution.

Fig. 17 illustrates the process by which the test program communicates with the target program. The Korean Requirement Analyzer (KRA) internally generates script code and

sends it to the target program according to the order of test cases. It also receives response scripts for each test case and stores them internally.

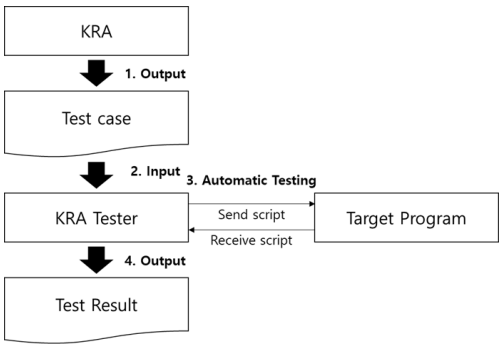


FIGURE 17. Automatic test execution process.

Fig. 18 shows the test report. The test report is also implemented as an XMI code. Test Steps mean the test items. Test Send Script is the script sent to the target program. The Expected Result is the anticipated outcome. Expected Receive Script is the script that is expected to be received from the target program. Actual Receive Script is the exact script received from the target program. Status means the verdict of the test case. Notes are for any additional information.

	Test Steps	Test Send Script	Expected Results	Expected Receive Script	Actual Receive Script	Status (Pass/Fail)	Notes
1	• ID/Password is entered. • ID/Password is invalid. • The login session expires.	• inputIDPW • invalidIDPW • expiredLOGIN	• System re-requests login information.	• rerequireLOGIN	• rerequireLOGIN	Pass	
2	• ID/Password is entered. • ID/Password is invalid.	• inputIDPW • invalidIDPW	• System re-requests login information.	• rerequireLOGIN	• rerequireLOGIN	Pass	
n							

FIGURE 18. Test result.

IV. EXPERIMENT AND RESULT

In this chapter, we conducted experiments to validate the proposed method by implementing a Korean requirement analyzer.

Fig. 19 illustrates the components of the Korean requirement analyzer, comprising an environment, libraries, and modules. The environment and libraries are external elements required for setting up the experimental environment. Ubuntu is a Linux operating system. Apache (PHP) is a web server.

Python is the development language used to operate Mecab-ko. Java is the development language used to operate elements of the Exobrain API and modules. Mecab-ko [17] is a Korean morphological analysis library. Exobrain [14] is a web-based API that supports syntactic and semantic role analysis. The modules represent the results of the processes proposed in this paper, which were implemented using the Java programming language.

The Sentence Analyzer is the Result of Steps 1-3. The Cause-Effect Graph Generator is the Result of Step 4. The Decision Table Generator is the Result of Step 5. The Test Case Generator is the Result of Step 7. The Test Runner is the Result of Step 8. Requirements, the integrated C3Tree model, the cause-effect graph Model, the decision table model, the test case model, and the test result model are all text files that are input to or output from each module.

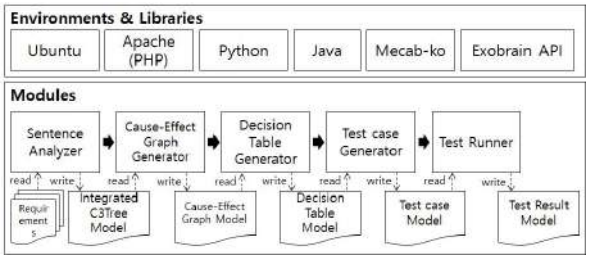


FIGURE 19. The whole environment of the Korean Requirement Analyzer.

A. EXPERIMENT DATA

In this section, we explain our approach using the example of a user communication system with 78 requirements specifications.

TABLE 11. The part of requirement sentences of the user communication system.

Language	Requirement Sentence
Korean	1. 얻은 아이디/비밀번호가 유효하지 않을 시 로그인 정보 재 요청한다. 2. 아이디/비밀번호가 유효하다면 사용자 개인 화면을 보여 준다. … 78. …
English	1. If the ID/password you obtained is invalid, re-request the login information. 2. If the ID/password is valid, the user's personal screen is displayed. … 78. ...

Table 11 shows some of the 78 functional requirements used in the experimental environment. The requirements are created as a single text file and input to the Sentence Analyzer. Sentence Analyzer automatically generates test cases from input requirements.

B. THE PROCEDURE OF OUR ANALYZER

With these 78 requirements, we execute the requirement analyzer as follows:

1) STEPS 1-3

When a requirement sentence is input, the C3Tree model is automatically generated, and the XMI code for the model is output at the top of the C3Tree model. Fig. 20 shows the result of developing the C3Tree model.

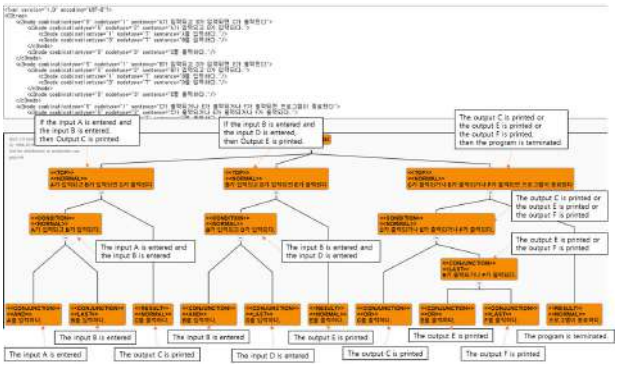


FIGURE 20. The generated C3Tree model (steps 1-3).

2) STEP 4

The cause-and-effect graph is automatically generated from the C3Tree model, and the XMI code for the graph is output at the top of the cause-and-effect graph. Fig. 21 shows an example of the cause-and-effect graph generation result.

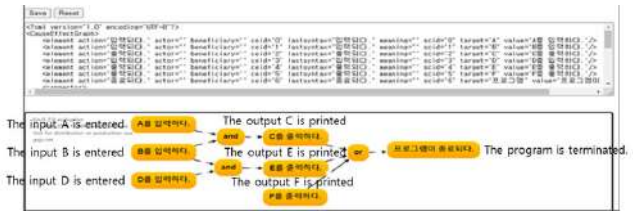


FIGURE 21. The generated cause-and-effect graph (step 4).

3) STEPS 5-6

The decision table is automatically generated from the cause-effect graph, and the XMI code of the graph is output at the top of the decision table. Fig. 22 shows a decision table example generated from a cause-and-effect graph.

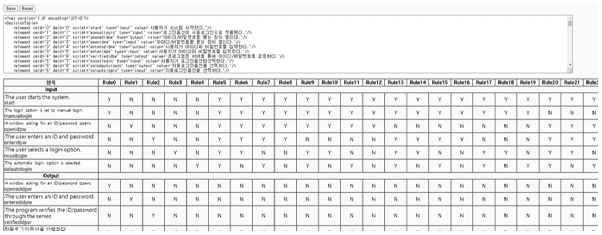
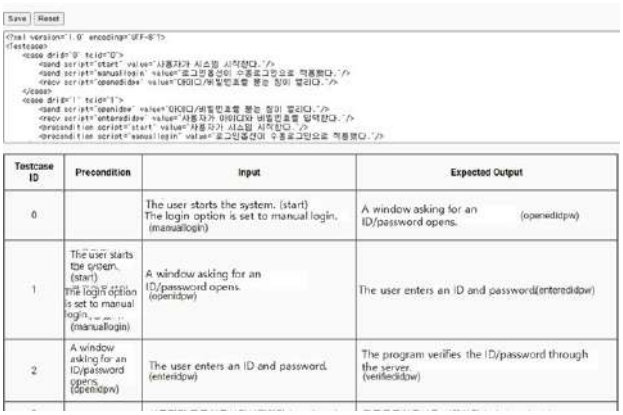


FIGURE 22. The generated decision table (step 5-6).

4) STEP 7

The Test Case is automatically generated from the decision table, and the XMI code of the graph is output at the top of the test case. Fig. 23 shows a test case example generated from a decision table.



Testcase ID	Precondition	Input	Expected Output
0		The user starts the system. (start) The login option is set to manual login. (manuallogin)	A window asking for an ID/password opens. (openeditpw)
1	The user starts the system. (start) the login option is set to manual login. (manuallogin)	A window asking for an ID/password opens. (openidpw)	The user enters an ID and password(enteredidpw)
2	A window asking for an ID/password opens. (openidpw)	The user enters an ID and password. (enteridpw)	The program verifies the ID/password through the server. (verifiedidpw)

FIGURE 23. The generated test case (step 7).

C. EXPERIMENTAL RESULT

We got the following results with 78 requirements of the user communication system. The sentence structure of 76 out of 79 was analyzed correctly (76/79, 96%). Among the sentences analyzed correctly, 24 sentences contained causes and effects (24/75, 32%). Sentences that do not contain cause and effect (52/76, 68%) are not incorrect analysis results. Our study analyzes whether a sentence contains cause clauses and effect clauses. Only sentences that contain cause clauses and effect clauses are transformed into test cases. If a sentence does not contain cause and effect, KRA asks the user to re-enter it. C3Tree models, cause-effect graphs, decision tables, test cases, and test reports were all analyzed correctly from the sentences containing causes and effects (24/24, 100%). Table 12 shows the experimental results.

As a result, 96% of the test cases generated from sentences containing cause and effect are 100%.

In the software engineering field, it isn't easy to generate test cases from Korean natural language requirements. Our research generates 32% of the total requirements.

Additionally, we compare our study with previous research. Table 13 presents the comparisons of existing research. Research 1 [2] can input natural language sentences, but cannot measure the similarity between sentences. Research 2 [3] can only input structured sentences. Research 3 [4] and 4 [5] can input non-structured sentences and measure similarity, but only accept English input and do not provide a requirement simplification process. Research 5 can analyze Korean, but only simple sentences are supported. Research 6 is our research. It analyzes Korean natural language sentences, traces the simplification process of sentences, and compares the similarity between sentences.

Our method requires that a complex sentence include a cause clause and a result clause. A natural language sentence containing a cause clause and a result clause can be automatically converted into a correct cause-effect graph. Moreover, the automatically converted cause-and-effect graph can be grafted onto other studies and used. However, actual requirement documents use various natural language formats. There can always be requirement sentences that do not contain cause clauses and result clauses. For example, "If the notepad program does not run, an execution failure notification window opens." is a sentence that includes a cause and an effect. "The user closes the notepad program by pressing the

X button." is a sentence that does not include a cause and an effect.

TABLE 12. Experiment result.

	Experimental Category	Result # of Requirement Sentences (Correct/Total)	Result %
# of requirements	Total input requirements	79	-
Analyzed requirements	Correctly analyzed requirements	76/79	96
With/without causes/effects	Requirements without cause and effect	52/76	68
	Requirements with cause and effect	24/76	32
Test cases generation with causes/effects of requirements (24)	C3Tree model generation success rate	24/24	100
	Cause-Effect graph generation success rate	24/24	100
	Decision table generation success rate	24/24	100
	Test case generation success rate	24/24	100
	Test result generation success rate	24/24	100

Our test case generation begins with identifying the smallest functional unit in the requirements. Table 14 shows the results of analyzing requirements sentences using generative AI with Chain-of-Thought prompting. The smallest functional unit in the sentence was identified, and the Identify/AND/OR/NOT relationships in the sentence were identified. The test data comprise the 24 sentences for which test cases were successfully generated, as shown in Table 12. The value of each item means the probability of the sentence being correctly generated. Because we are software engineers, we must verify and validate them for accuracy of test case. Existing research on cause-and-effect graph generation does not support the complete automation of this process. Recent popular generative AIs fail to create perfect cause-and-effect graphs because they incompletely identify the most minor functional units.

Table 15 shows the results of generating typical test cases using generative AI with Chain-of-Thought prompting. All generative AIs failed to generate accurate test cases, instead generating test cases for detailed functions not mentioned in the requirements. This was sometimes accurate and occasionally misleading.

V. CONCLUSION AND FUTURE RESEARCH

We aim to create test cases from requirements written in Korean automatically. Our requirement engineering approach

TABLE 13. Comparison of related research.

Function	Research 1	2	3	4	5	6
Natural Language Requirement Analysis (Korean)	X	X	X	X	O	O
Input Informal Requirement Specifications	O	X	O	O	O	O
Sentence Similarity Measurement	X	X	O	X	X	O
Requirements Traceability	X	X	X	X	X	O
Simplify Complex Requirement	X	X	X	X	X	O
Extract the Incomplete Requirement Sentence	X	X	X	X	X	O

TABLE 14. Generative AI analysis results using 23 sentences.

	Identifying the smallest unit of function	Generate logical connections
ChatGPT	0.73	0.86
Claude	0.78	0.60
Gemini	0.60	0.73
Copilot	0.69	0.60
Our research (standard)	1.00	1.00

TABLE 15. Test case generation results using generative AI.

	Generate accurate test cases.	Accurate traceability between models and requirements	Add functions not mentioned in the requirements	Drawing accurate graphs
ChatGPT	X	O	O	X
Claude	X	O	O	O
Gemini	X	O	O	X
Copilot	X	O	O	X
Our research	O	O	X	O

focuses on natural language-based Korean requirement specifications. We define the C3Tree Model for analyzing the requirements of complex sentences. In these models, leaf nodes with similar meanings are grouped using the Jaccard similarity mechanism and theta role comparison. In the next step, we transform the C3Tree models into cause-effect graphs, which are automatically turned into decision tables. We manually write test commands within this decision table. Finally, we can automatically generate test cases by extracting cause-and-effect and decision tables.

As a result, out of 78 sentences, 75 were accurately analyzed, and 23 sentences containing causes and effects were accurately transformed into C3Tree models. The remaining 52 sentences require sentence rewriting. All C3Tree models were successfully converted into test cases, and testing was performed based on the test cases to generate test reports.

Our current research investigates sentence analysis methods using AI-based learning and generates test cases for evaluation and assessment. Finally, we will compare which

method generates the test case more precisely between our approach and the AI-based approach.

REFERENCES

[1] O. S. Kwon and S. N. Hong, "Effective iterative testing based on log," in *Proc. Kor. Soc. MGMT. Info. Sys. Fall Conf.*, Nov. 2009, pp. 685–690.

[2] R. Elghondakly, S. Moussa, and N. Badr, "Waterfall and agile requirements-based model for automated test cases generation," in *Proc. IEEE 7th Int. Conf. Intell. Comput. Inf. Syst. (ICICIS)*, Dec. 2015, pp. 607–612, doi: 10.1109/INTELCIS.2015.7397285.

[3] H. Zheng, J. Feng, W. Miao, and G. Pu, "Generating test cases from requirements: A case study in railway control system domain," in *Proc. Int. Symp. Theor. Aspects Softw. Eng. (TASE)*, Aug. 2021, pp. 183–190, doi: 10.1109/tase52547.2021.00029.

[4] C. Wang, F. Pastore, A. Goknil, and L. C. Briand, "Automatic generation of acceptance test cases from use case specifications," *IEEE Trans. Softw. Eng.*, vol. 48, no. 2, pp. 585–616, May 2020, doi: 10.1109/TSE.2020.2998503.

[5] A. Ansari, M. B. Shagufta, A. S. Fatima, and S. Tehreem, "Constructing test cases using natural language processing," in *Proc. 3rd Int. Conf. Adv. Electr., Electron., Inf., Commun. Bio-Inform. (AEEICB)*, Feb. 2017, pp. 95–99, doi: 10.1109/AEEICB.2017.7972390.

[6] H. J. Han, K. Chung, and K. Choi, "Generating test cases and scripts from requirements in controlled language," *Proc. KIPS Trans. Softw. Data Eng.*, vol. 8, no. 8, pp. 331–342, Aug. 2019, doi: 10.3745/KTSDE.2019.8.8.331.

[7] H. Tufail, M. F. Masood, B. Zeb, F. Azam, and M. W. Anwar, "A systematic review of requirement traceability techniques and tools," in *Proc. 2nd Int. Conf. Syst. Rel. Saf. (ICSRS)*, Dec. 2017, pp. 450–454, doi: 10.1109/ICSRS.2017.8272863.

[8] W. S. Jang and R. Y. C. Kim, "Automatic generation mechanism of cause-effect graph with informal requirement specification based on the Korean language," *Appl. Sci.*, vol. 11, no. 24, p. 11775, Dec. 2021, doi: 10.3390/app112411775.

[9] W. S. Jang and R. Y. C. Kim, "Metamodeling approach for decision table generation from cause-effect graph at the intermediate stages of automatic test case generation," in *Proc. Smart Media App. Conf.*, Oct. 2022, pp. 26–28.

[10] J. M. Ha, "A contrastive study on Korean conditional connective ending and Chinese conditional conjunction expression," M.S. thesis, Dept. Korean Lang. Literature, Kyunghee Univ., South Korea, 2007.

[11] K. Kim, "Comparative study on conjoined sentence between modern Mongolian and Korean," *Kor. Assoc. Mong. Stud.*, vol. 27, pp. 151–186, Mar. 2009, doi: 10.17292/kams.2009..27.006.

[12] J. M. Cho, Y. H. Cho, and G. C. Kim, "A corpus formalization for extracting the syntactic relations," in *Proc. Annu. Conf. Hum. Lang. Tech.*, Oct. 1996, pp. 207–215.

[13] J. Paul, "The distribution of the flora in the Alpine zone.1," *New Phytologist.*, vol. 11, no. 2, pp. 37–50, Feb. 1912.

[14] S. Lim, M. Kwon, J. Kim, and H. Kim, "Korean proposition bank guidelines for exobrain," in *Proc. Annu. Conf. Hum. Lang. Tech.*, Oct. 2015, pp. 250–254.

[15] Object Manage. Group. (2025). *MDA Guide Revision 2.0*. [Online]. Available: https://www.omg.org/cgi-bin/doc?ormsc/14-06-01

[16] F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev, and P. Valduriez, "ATL: A QVT-like transformation language," in *Proc. 21st ACM SIGPLAN Symp. Object-Oriented Program. Syst., Lang., Appl.*, Oct. 2006, pp. 719–720, doi: 10.1145/1176617.1176691.

[17] W. Lee and Y. Yu. (2025). *Mecab-ko*. [Online]. Available: https://bitbucket.org/eunjeon/mecab-ko

WOOSUNG JANG received the M.S. and Ph.D. degrees in software engineering from Hongik University, South Korea, in 2022. He is currently an Adjunct Professor with Hongik University. His research interests include requirements analysis, Korean requirements modeling, model-based testing, metamodeling, and embedded programming.

R. YOUNG CHUL KIM received the B.S. degree in computer science from Hongik University, South Korea, in 1985, and the Ph.D. degree in software engineering from the Department of Computer Science, Illinois Institute of Technology (IIT), USA, in 2000. He is currently a Professor with Hongik University. His research interests include test maturity models, model-based testing, metamodeling, software process models, software visualization, and software validation of reinforced learning.